

Chương này trình bày các giải pháp kỹ thuật cốt lõi và đóng góp nổi bật trong quá trình thiết kế, xây dựng hệ thống WoodCert Auction. Các giải pháp tập trung giải quyết các bài toán về hiệu năng thời gian thực, tính nhất quán tài chính, sự công bằng trong đấu giá và bảo toàn tính toàn vẹn của dữ liệu thẩm định nguồn gốc gỗ. Đây là các giải pháp được thiết kế và hiện thực hóa trực tiếp dựa trên mã nguồn thực tế của dự án.

0.1 Giải pháp quản lý trạng thái phiên đấu giá lai và cơ chế chống đặt giá giây cuối

0.1.1 Giới thiệu bài toán và khó khăn

Đấu giá trực tuyến thời gian thực đặt ra hai thách thức kỹ thuật lớn đối với hệ thống phần mềm: tính đồng thời (Concurrency) và khả năng chịu tải (Scalability).

Trong những phút hoặc giây cuối cùng của một phiên đấu giá sôi nổi, lượng yêu cầu đặt giá (bid requests) gửi từ trình duyệt của nhiều người tham gia có xu hướng tăng đột biến. Nếu hệ thống sử dụng cơ sở dữ liệu quan hệ truyền thống như MySQL làm nguồn trạng thái duy nhất để xử lý trực tiếp, mỗi lượt đặt giá sẽ yêu cầu thực hiện khóa dòng hoặc khóa bảng để kiểm tra các điều kiện ràng buộc như: giá đặt mới phải lớn hơn giá hiện tại cộng với bước giá tối thiểu, phiên đấu giá phải còn trong thời gian hiệu lực và người dùng phải có tư cách tham gia hợp lệ. Các thao tác khóa dữ liệu đồng thời này sẽ gây ra hiện tượng nghẽn cổ chai cơ sở dữ liệu, làm tăng đáng kể độ trễ phản hồi, thậm chí dẫn đến lỗi tranh chấp khóa hoặc tình trạng race condition làm mất mát hoặc ghi đè các lượt đặt giá hợp lệ.

Ngoài ra, hệ thống còn phải đối mặt với rủi ro nghiệp vụ đặt giá giây cuối (Sniping). Đây là hành vi mà người tham gia cố tình chờ đến những giây cuối cùng của phiên đấu giá để đưa ra mức giá cao hơn một chút nhằm không cho những người tham gia khác cơ hội phản hồi. Hành vi này làm giảm tính cạnh tranh công bằng của sàn đấu giá và làm hạn chế khả năng đạt được giá trị chốt phiên của các tài sản gỗ có chứng thư.

Cuối cùng, việc xử lý sự kiện chính xác (Event Ordering) yêu cầu các lượt đặt giá phải được xử lý theo thứ tự tiếp nhận tại phía máy chủ, tránh trường hợp một lượt đặt giá đến sau lại được ghi nhận trước do sự bất đồng bộ trong luồng xử lý đa luồng của máy chủ.

0.1.2 Giải pháp kỹ thuật

Để giải quyết các khó khăn trên, trong giải pháp thiết kế của hệ thống, WoodCert Auction đã hiện thực hóa cơ chế quản lý trạng thái lai (Hybrid State Management) phối hợp giữa Redis và MySQL, kết hợp với giải thuật tự động gia hạn thời gian

(Auto-extension) ngay tại tầng xử lý bộ nhớ trong.

a, Tách biệt vai trò lưu trữ giữa Redis và MySQL

Hệ thống phân định ranh giới lưu trữ rõ ràng nhằm tối ưu hóa hiệu năng và độ tin cậy:

- **Redis (Nguồn trạng thái runtime chính):** Khi phiên đấu giá ở trạng thái hoạt động (ACTIVE), trong giải pháp quản lý của hệ thống, Redis đóng vai trò nguồn trạng thái chính để xử lý các yêu cầu đặt giá thời gian thực, trong khi MySQL vẫn là nơi lưu trữ bền vững của toàn hệ thống. Dữ liệu trạng thái phiên đấu giá và danh sách người tham gia đóng băng tiền cọc được lưu trên bộ nhớ trong của Redis dưới cấu trúc dữ liệu Hash (`auction:session:{id}:state`) và Set (`auction:session:{id}:bidders`).
- **MySQL (Nguồn lưu trữ bền vững):** MySQL lưu trữ dữ liệu bền vững của toàn hệ thống, gồm trạng thái phiên trước khi bắt đầu (WAITING), các trạng thái kết thúc (ENDED_SUCCESS, ENDED_FAILED, CANCELED), snapshot của phiên và lịch sử đặt giá. Sau khi lượt đặt giá được Redis chấp nhận, mã nguồn gọi tuần tự các phương thức ghi lịch sử và đồng bộ snapshot sang MySQL; các lời gọi này không chạy bằng @Async, nhưng được xử lý theo cơ chế best-effort để lỗi MySQL không làm đảo ngược kết quả đã được chấp nhận trên Redis.

b, Xử lý nguyên tử bằng Redis Lua Script

Trong giải pháp xử lý đặt giá thời gian thực của hệ thống, các logic kiểm tra và cập nhật được đóng gói trong một Redis Lua script. Vì Redis chạy đơn luồng (single-threaded), việc thực thi Lua script giúp hỗ trợ bảo đảm tính nguyên tử trong phạm vi một Redis instance. Các bước xử lý trong Lua script bao gồm:

1. **Kiểm tra thời gian:** So sánh thời gian máy chủ hiện tại (`nowMs`) với thời gian kết thúc phiên (`endTimeEpochMs`). Nếu phiên đã quá hạn, trả về trạng thái từ chối ENDED.
2. **Kiểm tra tư cách:** Xác thực sự tồn tại của thành viên trong Set đặt cọc bằng lệnh `SISMEMBER`; hệ thống trả về `NOT_REGISTERED` nếu không hợp lệ.
3. **Chống tự đặt giá:** So sánh người đặt giá hiện tại với người đang giữ giá cao nhất (`highestBidderId`). Nếu trùng nhau, chặn yêu cầu tự nâng giá của chính mình và trả về `SELF_BID`.
4. **Kiểm tra giá đặt tối thiểu:** Xác thực số tiền đặt mới phải lớn hơn hoặc bằng giá hiện tại cộng với bước giá tối thiểu (`currentPrice + stepPrice`). Nếu không thỏa mãn, trả về `LOW`.

5. **Cập nhật trạng thái:** Ghi nhận giá mới (`currentPrice`), người dẫn đầu (`highestBidderId`) và mã vết đặt giá (`highestBidTraceId`).
6. **Cơ chế tự động gia hạn (Auto-extension):** Tính toán thời gian còn lại của phiên (`endTimeEpochMs - nowMs`). Nếu thời gian còn lại nhỏ hơn hoặc bằng ngưỡng cấu hình (anti-sniper threshold, mặc định là 30 giây), hệ thống tự động cộng thêm thời gian gia hạn (anti-sniper extension, mặc định là 60 giây) vào thời gian kết thúc của phiên và cập nhật lại Redis Hash.

c, Phát sự kiện thời gian thực và lưu bền vững theo cơ chế best-effort

Sau khi Redis Lua script trả về kết quả thành công (OK), máy chủ thực hiện phát sự kiện `NEW_BID` chứa giá mới và thời gian kết thúc mới thông qua WebSocket STOMP tới topic `/topic/auctions/{id}`.

Sau bước broadcast, việc ghi nhận lịch sử đặt giá vào bảng `bids` và đồng bộ snapshot phiên đấu giá về MySQL được thực hiện tuần tự qua hai phương thức phụ trợ (`persistBidBestEffort` và `syncSessionSnapshotBestEffort`). Các phương thức này được bọc trong khối `try-catch` để xử lý ngoại lệ và ghi log lỗi tương ứng (`event=accepted_bid_not_persisted`, `event=accepted_bid_snapshot_not_synced`).

Trong cơ chế này, kết quả đặt giá đã thành công trên Redis không bị rollback nếu bước ghi MySQL gặp lỗi. Đánh đổi là trạng thái runtime trên Redis và dữ liệu bền vững trên MySQL có thể tạm thời sai lệch; khi đó giá hiện tại trong phòng đấu giá có thể khác lịch sử đọc từ MySQL. `AuctionSessionScheduler` hỗ trợ đồng bộ lại snapshot phiên theo chu kỳ, nhưng không tái tạo được bản ghi lịch sử `bids` đã thất bại. Vì vậy, log cảnh báo là căn cứ vận hành để phát hiện và xử lý trường hợp mất vết lịch sử.

0.1.3 Kết quả đạt được

- **Kiểm soát tranh chấp đồng thời:** Lua script gom các bước kiểm tra và cập nhật giá vào một thao tác nguyên tử trên Redis, nhờ đó hạn chế race condition trong phạm vi một Redis instance. Đề án chưa thực hiện benchmark tải để kết luận định lượng về độ trễ hoặc thông lượng của API đặt giá.
- **Hạn chế rủi ro đầu cơ và sniping:** Cơ chế auto-extension giúp giảm lợi thế của hành vi sniping và tăng cơ hội phản hồi công bằng cho các bidder khác. Điều này góp phần cải thiện giá trị chốt phiên của tài sản gõ đấu giá, hỗ trợ tính công bằng cho sàn giao dịch.
- **Phản hồi giao diện theo sự kiện:** WebSocket giúp các máy khách nhận giá và thời gian kết thúc mới sau khi Redis chấp nhận lượt đặt giá. Tính nhất quán

giữa Redis và MySQL là nhất quán cuối cùng theo cơ chế best-effort, không phải giao dịch nguyên tử xuyên hai hệ lưu trữ.

0.2 Quy trình xử lý ví và đặt cọc an toàn, lũy đẳng và tự động

0.2.1 Giới thiệu bài toán và khó khăn

Trong các hệ thống đấu giá trực tuyến, quy trình xử lý tài chính liên quan đến tiền đặt cọc là một trong những thành phần nhạy cảm và phức tạp. Nhằm hỗ trợ bảo đảm nghĩa vụ tham gia và thanh toán của các thành viên, WoodCert Auction áp dụng cơ chế đặt cọc (deposit). Trạng thái tham gia được chuyển từ FROZEN khi đăng ký sang REFUNDED đối với người không thắng hoặc DEDUCTED đối với người thắng để căn trừ vào giá trị đơn hàng. Các biến động ví tương ứng được ghi nhận bằng các loại giao dịch đóng băng, giải phóng và khấu trừ tiền cọc.

Thách thức kỹ thuật trong quy trình này là hạn chế hiện tượng xử lý trùng lặp (Duplicate Processing). Do các sự cố mạng hoặc độ trễ từ phía máy khách, người dùng có thể gửi yêu cầu thanh toán hoặc đặt cọc nhiều lần liên tiếp, hoặc hệ thống thực hiện cơ chế gửi lại yêu cầu (retry) khi gặp lỗi mạng. Nếu không được kiểm soát tốt, việc này có khả năng dẫn đến lỗi trừ tiền trùng lặp.

Vấn đề tiếp theo là khả năng hỗ trợ khôi phục sau sự cố (Fault Tolerance) và hỗ trợ duy trì tính nhất quán dữ liệu (Data Consistency). Các giao dịch tài chính thường đi qua nhiều module (Ví, Phiên đấu giá, Đơn hàng) và chịu sự điều phối của các tiến trình chạy nền. Nếu máy chủ bị sập đột ngột hoặc mất kết nối cơ sở dữ liệu giữa chừng khi đang xử lý đóng phiên đấu giá, hệ thống có thể rơi vào trạng thái không nhất quán (ví dụ: đã giải phóng cọc nhưng chưa cập nhật trạng thái đơn hàng, hoặc người thắng bị trừ tiền nhưng đơn hàng chưa được khởi tạo).

Bên cạnh đó, hệ thống còn phải đối phó với rủi ro nghiệp vụ người thắng không thanh toán. Sau khi phiên đấu giá kết thúc thành công, nếu người thắng cuộc cố tình từ chối thanh toán số tiền còn lại trong thời hạn thanh toán theo cấu hình, hệ thống thực hiện quy trình tịch thu cọc làm hình phạt nhằm bù đắp chi phí cơ hội cho người bán và vận hành sàn.

0.2.2 Giải pháp kỹ thuật

Trong giải pháp xử lý, WoodCert Auction giải quyết các bài toán trên thông qua cơ chế giao dịch lũy đẳng (Idempotent Mutation), quản lý khóa tối ưu ở tầng lưu trữ, và sự phối hợp đồng bộ giữa các Scheduler chạy nền.

a, Thiết kế giao dịch ví lũy đẳng với Operation Key

Các biến động số dư ví (available_balance, frozen_balance) trong giải pháp tài chính của hệ thống đều được thực hiện qua phương thức điều phối

`executeIdempotentMutation`.

- **Đăng ký Operation Key:** Trước khi thực hiện các thay đổi sổ dư, hệ thống tạo một khóa lũy đẳng duy nhất (`operationKey`) theo định dạng riêng của từng nghiệp vụ, ví dụ `auction:register:freeze:{auctionId}:{userId}`. Khóa này được lưu vào bảng `wallet_operations` trước khi giao dịch ví bắt đầu.
- **Kiểm soát trùng lặp:** Khi một yêu cầu trùng lặp hoặc yêu cầu retry gửi tới, hệ thống kiểm tra sự tồn tại của `operationKey`. Nếu khóa đã tồn tại ở trạng thái thành công, hệ thống sẽ trả về kết quả và bỏ qua phần xử lý phía sau, giúp ngăn chặn hiệu quả lỗi xử lý trùng lặp tài chính trong phạm vi cơ chế hiện thực của Operation Key.

b, Kiểm soát tranh chấp đồng thời và đồng bộ Transaction

- **Khóa tối ưu (Optimistic Locking):** Bảng ví (`wallets`) sử dụng thuộc tính phiên bản `@Version` để thực hiện khóa tối ưu. Khi có nhiều luồng cố gắng cập nhật sổ dư ví của cùng một tài khoản cùng lúc, JPA sẽ phát hiện xung đột phiên bản và ném ra ngoại lệ `WALLET_CONCURRENT_MODIFICATION`, giúp bảo vệ dữ liệu ví không bị ghi đè sai lệch.
- **Spring Transaction Synchronization:** Trạng thái của `WalletOperation` được chuyển sang thành công (`markSuccess`) tại sự kiện `afterCommit()` của Spring Transaction Manager. Nếu các thao tác ghi dữ liệu xuống MySQL bị lỗi và dẫn đến rollback, trạng thái của khóa lũy đẳng sẽ tự động chuyển về thất bại (`markFailed`) thông qua hook `afterCompletion(status)`, giúp hỗ trợ duy trì tính nhất quán dữ liệu giữa bảng sổ dư ví và bảng nhật ký giao dịch.

c, Tự động hóa xử lý bằng Scheduler chạy nền

Hệ thống sử dụng các tác vụ nền (Schedulers) để tự động hóa các quy trình cọc và xử lý vi phạm:

- **Tác vụ đóng phiên đấu giá:** Quét định kỳ các phiên đấu giá đến hạn. Khi phiên kết thúc thành công, tác vụ nền khấu trừ tiền cọc của người thắng, giải phóng tiền cọc của những người còn lại và tạo đơn hàng chờ thanh toán. Việc quyết toán từng người tham gia được thực hiện trong giao dịch `REQUIRES_NEW` riêng, do đó không phải một giao dịch nguyên tử duy nhất bao trùm toàn bộ danh sách người tham gia.
- **Tác vụ xử lý quá hạn thanh toán:** Tác vụ nền quét các đơn hàng quá hạn thanh toán theo thời gian cấu hình. Nếu người mua không thanh toán phần

còn lại của đơn hàng đúng hạn, hệ thống tự động hủy đơn và thực hiện tịch thu tiền đặt cọc đã capture, phân phối theo tỷ lệ phân chia được cấu hình trong hệ thống (đọc từ thuộc tính `forfeitedDepositPlatformRate` trong file cấu hình tài chính) cho nền tảng và người bán.

0.2.3 Kết quả đạt được

- **Giảm thiểu rủi ro giao dịch:** Cơ chế `WalletOperation` lũy đẳng kết hợp khóa tối ưu giúp kiểm soát tốt các lỗi tài chính phát sinh do yêu cầu trùng lặp hoặc truy cập đồng thời, hỗ trợ duy trì tính nhất quán tài chính trong phạm vi cơ chế hiện thực.
- **Tự động hóa luồng phạt cọc:** Luồng nghiệp vụ phạt cọc khi người thắng bùng thanh toán được thực thi chính xác và tự động thông qua tác vụ nền mà không cần sự can thiệp thủ công của quản trị viên, góp phần bảo vệ quyền lợi kinh tế của người bán.
- **Bảo vệ người mua khi người bán quá hạn giao hàng:** Fulfillment được gán `shipment_deadline` sau khi đơn được thanh toán, mặc định là 72 giờ. Scheduler quét các fulfillment `PENDING_SHIPMENT` quá hạn, khóa bản ghi, hủy đơn, hoàn lại toàn bộ tiền cọc cùng phần tiền đã thanh toán cho người mua và đưa sản phẩm về trạng thái có thể giao dịch. Hệ thống vẫn chưa áp dụng tiền cọc, phạt tài chính hoặc điểm uy tín đối với người bán vi phạm.

0.3 Cơ chế bảo toàn tính toàn vẹn của báo cáo thẩm định gỗ và chứng thư

0.3.1 Giới thiệu bài toán và khó khăn

Trong thị trường giao dịch và đấu giá gỗ quý, tính toàn vẹn của thông tin thẩm định nguồn gốc và chất lượng của sản phẩm là yếu tố quan trọng quyết định đến giá trị kinh tế của tài sản. Để giải quyết rủi ro nghiệp vụ sản phẩm không đúng mô tả hoặc sai lệch nguồn gốc, hệ thống yêu cầu một quy trình kiểm soát chất lượng chặt chẽ trước khi sản phẩm được đưa lên sàn đấu giá.

Khó khăn đặt ra ở đây là bảo vệ tính toàn vẹn của thông tin thẩm định sau khi đã được ban hành bởi Thẩm định viên (Appraiser). Nếu không có cơ chế bảo vệ, dữ liệu báo cáo thẩm định lưu trong cơ sở dữ liệu có khả năng bị sửa đổi do lỗi hỏng bảo mật, sự can thiệp trực tiếp từ quản trị viên hệ thống hoặc hành vi gian lận từ phía người bán nhằm nâng khống giá trị gỗ.

Bên cạnh đó, hệ thống yêu cầu về khả năng truy vết (Auditability) và đồng bộ thời gian (Time Synchronization). Mỗi báo cáo thẩm định khi được phê duyệt phải được gắn mốc thời gian máy chủ thống nhất và có khả năng đối chiếu lại thông tin gốc khi xảy ra tranh chấp phát sinh giữa người mua và người bán sau khi nhận

hàng.

0.3.2 Giải pháp kỹ thuật

Trong giải pháp bảo đảm tính toàn vẹn dữ liệu của hệ thống, WoodCert Auction giải quyết thách thức này bằng quy trình thẩm định đa trạng thái kết hợp với mã vân tay toàn vẹn (Integrity Fingerprint) sử dụng thuật toán băm mật mã SHA-256 cho từng báo cáo thẩm định.

a, Quy trình kiểm định và ban hành chứng thư

Sản phẩm gỗ muốn đấu giá phải đi qua máy trạng thái thẩm định khép kín: DRAFT → PENDING_APPRAISAL → UNDER_APPRAISAL → APPRAISED (hoặc REJECTED).

- Khi Thẩm định viên nhận yêu cầu, hệ thống khóa bản ghi liên quan ở tầng cơ sở dữ liệu để hạn chế nhiều người đồng thời nhận cùng một yêu cầu. Mã UUID tạm thời chỉ được dùng trong bước lưu báo cáo nhằm tạo bản ghi và lấy ID, không được cấp ngay tại thời điểm nhận yêu cầu.
- Sau khi Thẩm định viên hoàn thành các thông tin giám định, báo cáo được lưu và hệ thống sinh số chứng thư theo định dạng CERT-`{year}`-`{id}` (trong đó `year` là năm hiện tại và `id` là ID tăng tự động của báo cáo). Danh sách ảnh bằng chứng là dữ liệu có thể đính kèm, không phải điều kiện bắt buộc của mọi báo cáo.

b, Cơ chế sinh mã vân tay toàn vẹn SHA-256 (Integrity Fingerprint)

Để tạo dấu vân tay tham chiếu cho báo cáo tại thời điểm ban hành, hệ thống thực hiện các bước sau:

1. **Ghép nối dữ liệu (Payload Serialization):** Hệ thống tiến hành ghép nối các thuộc tính cốt lõi của báo cáo thẩm định thành một chuỗi văn bản theo một thứ tự cố định, ngăn cách bởi ký tự đặc biệt:

```
productId|appraiserId|verifiedMaterial|  
estimatedValue|isAuthentic|certificateCode|  
appraisedAt
```

(trong đó `appraisedAt` là mốc thời gian ghi nhận của máy chủ lúc phê duyệt báo cáo).

2. **Tính toán mã băm:** Chuỗi payload trên được băm bằng thuật toán băm mật mã SHA-256 để sinh ra một chuỗi Hex băm duy nhất dài 64 ký tự (Integrity Fingerprint) và được lưu trữ bền vững cùng báo cáo thẩm định trong MySQL.
3. **Lưu dấu vân tay:** Giá trị `integrityHash` được trả về cùng thông tin chứng

thư khi truy vấn. Phiên bản hiện tại chưa tự động tính lại payload và so sánh mã băm trong luồng đọc; vì vậy mã băm mới đóng vai trò dấu vân tay được lưu tại thời điểm phê duyệt, chưa phải cơ chế kiểm chứng thay đổi tự động.

0.3.3 Kết quả đạt được

- **Tạo dấu vân tay cho dữ liệu tại thời điểm phê duyệt:** SHA-256 tạo một giá trị tham chiếu ổn định từ payload cốt lõi của báo cáo. Muốn dùng giá trị này để phát hiện sửa đổi, hệ thống cần bổ sung bước tái tạo payload và đối chiếu khi đọc hoặc khi kiểm toán. Tính đúng đắn của nội dung giám định ban đầu vẫn phụ thuộc vào năng lực chuyên môn của Thẩm định viên và quy trình kiểm định thực tế.
- **Khả năng truy vết rõ ràng:** Mốc thời gian phê duyệt thống nhất dựa trên thời gian máy chủ cùng mã băm toàn vẹn tạo cơ sở kỹ thuật hỗ trợ truy vết và đối chiếu khi xảy ra tranh chấp chất lượng sau đấu giá.
- **Giới hạn xác thực:** Mã băm không phải chữ ký số, không xác minh danh tính người ký và không cung cấp khả năng chống chối bỏ. Do mã băm được lưu cùng cơ sở dữ liệu với báo cáo, cơ chế hiện tại cũng chưa độc lập trước người có quyền sửa đồng thời cả dữ liệu và giá trị băm.

0.4 Tổng kết chương

Chương 5 đã trình bày ba nhóm giải pháp được hiện thực hóa trong WoodCert Auction. Hybrid State Management kết hợp Redis Lua script và cơ chế tự động gia hạn để xử lý trạng thái phiên ACTIVE, đồng thời chấp nhận đánh đổi nhất quán cuối cùng với MySQL. Idempotent Wallet Mutation sử dụng WalletOperation, khóa dữ liệu và các giao dịch tách biệt để hạn chế xử lý tài chính trùng lặp; luồng giao nhận còn bổ sung deadline 72 giờ và hoàn tiền tự động cho người mua khi người bán không gửi hàng. Appraisal Integrity Fingerprint tạo dấu vân tay SHA-256 tại thời điểm phê duyệt báo cáo, nhưng phiên bản hiện tại chưa tự động đối chiếu lại và không thay thế chữ ký số. Các kết luận này phản ánh phạm vi mã nguồn và kiểm thử hiện có, không được hiểu là kết quả benchmark tải hoặc chứng nhận an toàn độc lập.